1. Write a Python program to solve the 8-puzzle problem.

Programiz
Python Online Compiler

main.py

```python
from collections import deque

goal = "123456780"
start = "724506831"   # change this (0 = blank)


def neighbors(state):
    i = state.index("0")
    r, c = divmod(i, 3)
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            j = nr * 3 + nc
            s = list(state)
            s[i], s[j] = s[j], s[i]
            yield "".join(s)


def solve(start):
    queue = deque([start])
    parent = {start: None}
    while queue:
```

Programiz
Python Online Compiler

Programiz PRO ›

main.py

Visualize | Run

```python
        state = queue.popleft()
        if state == goal:
            path = []
            while state:
                path.append(state)
                state = parent[state]
            return path[::-1]
        for n in neighbors(state):
            if n not in parent:
                parent[n] = state
                queue.append(n)
    return None


path = solve(start)
if path is None:
    print("No solution")
else:
    print("Moves:", len(path) - 1)
    for s in path:
        print(s[0:3], s[3:6], s[6:9], sep="\n")
        print()
```

Output

```
023
145
786

123
045
786

123
405
786

123
450
786

123
456
780
```

Edit with WPS Office

2. Write a Python program to solve the 8 queens problem.

main.py

```python
N = 8
solutions = []


def is_safe(board, row, col):
    for r in range(row):
        c = board[r]
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True


def solve(board, row):
    if row == N:
        solutions.append(board[:])
        return
    for col in range(N):
        if is_safe(board, row, col):
            board.append(col)
            solve(board, row + 1)
            board.pop()
```

main.py

Visualize Run

```python

def solve(board, row):
    if row == N:
        solutions.append(board[:])
        return
    for col in range(N):
        if is_safe(board, row, col):
            board.append(col)
            solve(board, row + 1)
            board.pop()


def print_board(board):
    for col in board:
        print(" ".join("Q" if c == col else "." for c in range(N)))
    print()


solve([], 0)
print("Total solutions:", len(solutions))
print("\nFirst solution:\n")
print_board(solutions[0])
```

Output

```
Total solutions: 92

First solution:

Q . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

3. Write a Python program for the water jug problem.

main.py

```python
from collections import deque

cap_a = 4      # capacity of jug A
cap_b = 3      # capacity of jug B
target = 2     # required amount of water in either jug


def next_states(a, b):
    return [
        ((cap_a, b), "Fill jug A"),
        ((a, cap_b), "Fill jug B"),
        ((0, b), "Empty jug A"),
        ((a, 0), "Empty jug B"),
        ((a - min(a, cap_b - b), b + min(a, cap_b - b)), "Pour A -> B"),
        ((a + min(b, cap_a - a), b - min(b, cap_a - a)), "Pour B -> A"),
    ]


def solve():
    start = (0, 0)
    queue = deque([start])
    parent = {start: None}
```

main.py

Visualize Run

Output

```python
        if target in state:
            path = []
            while parent[state]:
                prev, action = parent[state]
                path.append((action, state))
                state = prev
            return path[::-1]
        for new, action in next_states(*state):
            if new not in parent:
                parent[new] = (state, action)
                queue.append(new)
    return None


path = solve()
if path is None:
    print("No solution")
else:
    print(f"Solved in {len(path)} steps\n")
    print("Start: (A=0, B=0)")
    for action, (a, b) in path:
        print(f"{action:<12} -> (A={a}, B={b})")
```

```
Solved in 4 steps

Start: (A=0, B=0)
Fill jug B   -> (A=0, B=3)
Pour B -> A  -> (A=3, B=0)
Fill jug B   -> (A=3, B=3)
Pour B -> A  -> (A=4, B=2)
```

4. Write a Python program for the cryptarithmetic problem.

main.py

```python
from itertools import permutations

word1 = "SEND"
word2 = "MORE"
result = "MONEY"


def to_number(word, mapping):
    return int("".join(str(mapping[ch]) for ch in word))


def solve():
    letters = sorted(set(word1 + word2 + result))
    if len(letters) > 10:
        return None
    first_letters = {word1[0], word2[0], result[0]}

    for digits in permutations(range(10), len(letters)):
        mapping = dict(zip(letters, digits))
        if any(mapping[ch] == 0 for ch in first_letters):
            continue
        if to_number(word1, mapping) + to_number(word2, mapping) == to_number(result, mapping):
            return mapping
    return None


mapping = solve()
if mapping is None:
    print("No solution")
else:
    print("Letter mapping:", mapping)
    print()
    print(f"  {word1} = {to_number(word1, mapping)}")
    print(f"+ {word2} = {to_number(word2, mapping)}")
    print("-" * 14)
    print(f"{result} = {to_number(result, mapping)}")
```

Visualize | Run

Output

```
Letter mapping: {'D': 7, 'E': 5, 'M': 1, 'N': 6, 'O': 0, 'R': 8, 'S': 9, 'Y': 2}

  SEND = 9567
+ MORE = 1085
--------------
MONEY = 10652
```

Edit with WPS Office

5. Write a Python program for the missionaries and cannibals problem.

main.py

Visualize

Run

```python
from collections import deque

N = 3          # number of missionaries and cannibals
BOAT = 2       # boat capacity

# State = (missionaries_left, cannibals_left, boat_side)  boat_side: 0 = left, 1 = right
start = (N, N, 0)
goal = (0, 0, 1)


def is_valid(m, c):
    """Valid if counts are in range and missionaries are never outnumbered on a bank."""
    if not (0 <= m <= N and 0 <= c <= N):
        return False
    if m > 0 and c > m:                    # left bank unsafe
        return False
    if (N - m) > 0 and (N - c) > (N - m):    # right bank unsafe
        return False
    return True


def next_states(state):
```

Edit with WPS Office

Output

```
Solved in 11 crossings

Start: Left = 3M 3C | Right = 0M 0C
Move 0M 2C ->   Left = 3M 1C | Right = 0M 2C
Move 0M 1C <-   Left = 3M 2C | Right = 0M 1C
Move 0M 2C ->   Left = 3M 0C | Right = 0M 3C
Move 0M 1C <-   Left = 3M 1C | Right = 0M 2C
Move 2M 0C ->   Left = 1M 1C | Right = 2M 2C
Move 1M 1C <-   Left = 2M 2C | Right = 1M 1C
Move 2M 0C ->   Left = 0M 2C | Right = 3M 1C
Move 0M 1C <-   Left = 0M 3C | Right = 3M 0C
Move 0M 2C ->   Left = 0M 1C | Right = 3M 2C
Move 0M 1C <-   Left = 0M 2C | Right = 3M 1C
Move 0M 2C ->   Left = 0M 0C | Right = 3M 3C
```

main.py

Visualize | Run

```
def next_states(state):
    m, c, side = state
    sign = -1 if side == 0 else 1           # boat leaves left: people decrease on left
    for dm in range(BOAT + 1):
        for dc in range(BOAT + 1 - dm):
            if dm + dc == 0:
                continue
            nm, nc = m + sign * dm, c + sign * dc
            if is_valid(nm, nc):
                yield (nm, nc, 1 - side), (dm, dc)


def solve():
    queue = deque([start])
    parent = {start: None}
    while queue:
        state = queue.popleft()
        if state == goal:
            path = []
            while parent[state]:
                prev, move = parent[state]
                path.append((move, state))
```

Output

```
Solved in 11 crossings

Start: Left = 3M 3C | Right = 0M 0C
Move 0M 2C ->   Left = 3M 1C | Right = 0M 2C
Move 0M 1C <-   Left = 3M 2C | Right = 0M 1C
Move 0M 2C ->   Left = 3M 0C | Right = 0M 3C
Move 0M 1C <-   Left = 3M 1C | Right = 0M 2C
Move 2M 0C ->   Left = 1M 1C | Right = 2M 2C
Move 1M 1C <-   Left = 2M 2C | Right = 1M 1C
Move 2M 0C ->   Left = 0M 2C | Right = 3M 1C
Move 0M 1C <-   Left = 0M 3C | Right = 3M 0C
Move 0M 2C ->   Left = 0M 1C | Right = 3M 2C
Move 0M 1C <-   Left = 0M 2C | Right = 3M 1C
Move 0M 2C ->   Left = 0M 0C | Right = 3M 3C
```

Edit with WPS Office

main.py

Visualize

Run

```
                prev, move = parent[state]
                path.append((move, state))
                state = prev
            return path[::-1]
        for new, move in next_states(state):
            if new not in parent:
                parent[new] = (state, move)
                queue.append(new)
    return None


path = solve()
if path is None:
    print("No solution")
else:
    print(f"Solved in {len(path)} crossings\n")
    print(f"Start: Left = {N}M {N}C | Right = 0M 0C")
    side = 0
    for (dm, dc), (m, c, _) in path:
        direction = "->" if side == 0 else "<-"
        print(f"Move {dm}M {dc}C {direction}   Left = {m}M {c}C | Right = {N - m}M {N - c}C")
        side = 1 - side
```

Output

```
Solved in 11 crossings

Start: Left = 3M 3C | Right = 0M 0C
Move 0M 2C ->   Left = 3M 1C | Right = 0M 2C
Move 0M 1C <-   Left = 3M 2C | Right = 0M 1C
Move 0M 2C ->   Left = 3M 0C | Right = 0M 3C
Move 0M 1C <-   Left = 3M 1C | Right = 0M 2C
Move 2M 0C ->   Left = 1M 1C | Right = 2M 2C
Move 1M 1C <-   Left = 2M 2C | Right = 1M 1C
Move 2M 0C ->   Left = 0M 2C | Right = 3M 1C
Move 0M 1C <-   Left = 0M 3C | Right = 3M 0C
Move 0M 2C ->   Left = 0M 1C | Right = 3M 2C
Move 0M 1C <-   Left = 0M 2C | Right = 3M 1C
Move 0M 2C ->   Left = 0M 0C | Right = 3M 3C
```

Edit with WPS Office